Apple Keynote .key Format Analysis Report

> **Note**: This report records the format analysis process, empirical findings, and command usage.

- **Subject**: sample-deck.key (user-created theme, 465,397 bytes)
- **Environment**: macOS / Keynote 14.5 (Build 7045.0.17)
- **Method**: reverse-engineering toolchain + a byte-level decoder + the official protobuf schema (v14_5 definitions), cross-checked against Keynote itself; every conclusion was accepted only after the file opened in Keynote.
- **Report date**: 2026-10-16 (color-writing addendum: 2026-10-17)

---

1. Executive Summary

A .key file is a composite format that can be fully unpacked, semantically parsed, addressed page by page, and safely written back. The analysis covers seven layers:

Layer	Format	Status
Container	Zip archive (all entries ZIP_STORED)	Confirmed
Data	IWA: 0x00 + little-endian uint24 + Snappy blocks (possibly multiple chunks)	Confirmed
Segment	Sequence of IWAArchiveSegment messages (protobuf)	Confirmed
Semantics	Object types and contents (631 type mappings)	Confirmed
Reference graph	Complete Show → SlideNode → Slide → text/notes addressing chain	Confirmed
State switches	Page-level switches such as isSkipped	Confirmed
Object content	Table text/numbers, builds, theme and element colors	Working

Capabilities confirmed by opening the result in Keynote:

- Unpack and read every structure and media resource.
- **Identify every object semantically**: all 1,360 objects parsed, 0 failures, 0 unknown types.
- Extract real document text (titles, bodies, footers, layout names).
- Export a page outline: title, body, and presenter notes (a 21-page deck read completely).
- Replace text with equal-length or different-length strings; nested length chains are recalculated automatically.
- Batch-replace across every .iwa file (17 files, 17 footer occurrences in one run).
- Append presenter notes by page without rewriting existing notes.
- Skip or unskip selected pages with expressions such as 1-3,7 or all.
- Rewrite a complete title/body by following the reference chain to StorageArchive.text.
- Replace existing notes, either as a whole or via substring replacement.

- Audit note compliance (missing notes, missing content/design rationale).
- Replace media payloads by replacing same-named Zip entries.
- Write table text through the string table under Index/Tables/.
- Read and write builds (list builds, change duration and automatic playback).
- Run a combined audit over notes, skipped pages, builds, and tables.
- Write table numbers through scaled integers in cell_storage_buffer; Keynote reads back 0.777.
- Read/write colors: enumerate, set by index, change page backgrounds and shape fills; verified with Keynote and PNG pixels.
- Apply page-precise text color through a new CharacterStyle attached to table_char_style, without modifying shared styles.
- Add/delete slides through slideTree.slides, with cascading cleanup of owned objects.
- Add/delete shapes through owned_drawables and drawables_z_order, with cascading cleanup of storage/caption objects.
- Rewrite existing notes with set-note.

Structural editing: slide/shape insertion and deletion plus page-precise text color are working. Cross-version behavior still requires real-device verification.

---

2. Layer 1: Zip Container

A .key file is a Zip archive (Zip archive data, compression method=store). The sample has **60 entries** and no symlinks:

> Index/*.iwa 31 entries — document and slide data
> Data/*.jpg 24 entries — media
> Metadata/Properties.plist / DocumentIdentifier / BuildVersionHistory.plist
> preview.jpg / preview-web.jpg / preview-micro.jpg

All entries use ZIP_STORED (no compression), so re-packing does not need to handle compression differences.

---

3. Layer 2: IWA Data Format

> one .iwa = one or more concatenated chunks
> each chunk = [0x00] [compressed length, little-endian uint24] [Snappy data]

File	Size	Chunks	Decompressed
Document.iwa	7,751	1	14,705
Slide.iwa	1,480	1	3,004
DocumentStylesheet-3843.iwa	47,811	**5**	—

Key points: the length field is a **little-endian uint24**; one .iwa may contain multiple chunks; Snappy uses the **raw/block format** with no sNaPpY magic.

---

4. Layer 3: Segment Structure

This is the easiest layer to get wrong. Splitting with the assumption that every message has a varint length prefix fails immediately. The actual rule is:

```
after Snappy decompression (multiple chunks concatenated) = a sequence of IWAArchiveSegment

IWAArchiveSegment =
    [varint: byte length of ArchiveInfo]    ← only this part has a length prefix
    [ArchiveInfo]
    [object_0][object_1]...                 ← raw concatenation; boundaries come from lengths

ArchiveInfo = { field 1: identifier, field 2: message_infos[] }
MessageInfo = { field 1: type, field 2: version,
                field 3: length ★exact object byte length; the only boundary★,
                field 4/5: reference lists }
```

Only ArchiveInfo has a length prefix; objects are delimited by MessageInfo.length.

Verified example: the first ArchiveInfo in Slide.iwa is 60 bytes and its first MessageInfo.length is 180; the following raw bytes are exactly 180 bytes.

Full verification: 31 .iwa files parsed cleanly to the end, totaling **1,354 segments / 1,360 objects**, with zero failures.

Naming convention: the number in an .iwa filename is the file's ArchiveInfo.identifier (Document.iwa = 1, Metadata.iwa = 2, TemplateSlide-1610803-2.iwa = 1610803).

---

5. Layer 4: Object Semantics

A segment boundary is not enough; every object needs a type. Keynote identifies object types with the numeric MessageInfo.type. That mapping is not public, but the **v14_5** definitions of the official schema provide it.

Mapping sizes: ID_NAME_MAP = **631** entries; NAME_CLASS_MAP = **1,359** entries. Version alignment: Keynote **14.5** ⟷ schema **v14_5**.

Key types:

Type	Class	Meaning
1	KN.DocumentArchive	Document.iwa, identifier = 1
2	KN.ShowArchive	Metadata.iwa, identifier = 2
4	KN.SlideNodeArchive	slide-tree node

Type	Class	Meaning
5 / 6	KN.SlideArchive	slide body
9	KN.SlideStyleArchive	slide style
14	TSWP.TextualAttachmentArchive	textual attachment
15	KN.NoteArchive	presenter notes

Full semantic parse: all **1,360 objects** in the sample parsed successfully (0 failures / 0 unknown types):

```
360  TSD.StandinCaptionArchive
205  TSWP.StorageArchive          ← text container
149  TSWP.ParagraphStyleArchive
102  TST.CellStyleArchive
101  TSWP.ShapeInfoArchive
 95  KN.PlaceholderArchive
 41  TSWP.ShapeStyleArchive
 36  TSCH.ChartSeriesStyleArchive
 24  KN.SlideNodeArchive
 24  KN.SlideArchive              ← 24 slides
 24  TSD.GuideStorageArchive
 18  TSCH.ChartAxisStyleArchive
 18  KN.MotionBackgroundStyleArchive
```

---

6. Reference Graph: Connecting “Slide N”

Object types alone are not enough; the document structure lives in a graph of TSP.Reference values. The complete chain is:

```
KN.ShowArchive (field 3: slideTree)            ← in Index/Document.iwa
  └ KN.SlideTreeArchive.slides[]               (inline message, repeated TSP.Reference)
    └ KN.SlideNodeArchive.slide                (TSP.Reference)
      └ KN.SlideArchive
        ├ .titlePlaceholder / .bodyPlaceholder  (TSP.Reference)
        │   └ KN.PlaceholderArchive.super       ★inline TSWP.ShapeInfoArchive
        │     └ ShapeInfoArchive.owned_storage (TSP.Reference)
        │       └ TSWP.StorageArchive.text      (repeated string)
        ├ .owned_drawables[] → same shape; TSD.GroupArchive.children must recurse
        └ .note                                 (TSP.Reference)
          └ KN.NoteArchive.containedStorage  (TSP.Reference)
            └ TSWP.StorageArchive.text      ← presenter notes
```

Three traps, all cases where something looks like a reference but is actually inline:

1. ShowArchive.slideTree is an **inline message**; do not dereference .identifier on it.
2. A placeholder points to **KN.PlaceholderArchive**; its super (field 1) is the TSWP.ShapeInfoArchive.
3. Text hangs off **ShapeInfoArchive.owned_storage (field 4)**, not deprecated_storage (2) or text_flow (3).

Observed output (21-page sample):

```
--- Page 1 ---
  title: Sample deck
  body: A real build-vs-buy trade-off
  notes: present (185 chars)
--- Page 3 ---
  title: Sample section title
  body: ① System shape: pure example / pure Wiki / dual-track
  notes: present (145 chars)
```

Additional finding: all 21 pages have presenter notes that follow the 【Content basis】/【Design basis】 convention. This means notes can be checked programmatically, for example by finding pages with missing notes or invalid fields.

---

7. Text: Plain and Semantically Addressable

Text is stored in TSWP.StorageArchive; its text field is a **repeated string** (one element per paragraph). The sample has 74 non-empty text objects, for example:

Object ID	File	Content
1611013	TemplateSlide.iwa	Title text
1610829	TemplateSlide.iwa	Body level 1 / Body level 2 / … / Body level 5 (5 paragraphs)
6879323	TemplateSlide.iwa	© Sample copyright (footer)
6682201 segment	TemplateSlide-6682201.iwa	layout name Title – Top Aligned_Black Background

All text is plain UTF-8 with no encryption or obfuscation and can be located precisely by object ID.

7.1 Media (Data/)

Files under Data/ (sample: 21 mt-* and 22 st-* JPEGs) are referenced by **TSP.DataReference**, which has only an identifier field. The main referrer is KN.SlideNodeArchive.thumbnails (slide thumbnails). Content images hang off TSD.ImageArchive: data (11) / originalData (13) / thumbnailData (12) → TSP.DataReference → a file under Data/.

Key conclusion: media files have no digest or size check. TSP.DataMetadata has only a fallback_color field; there is no digest or size. Therefore:

Replacing media = replacing the bytes of the same-named Zip entry while keeping the filename unchanged, so references never break.

Verified by replacing a Data/mt-*.jpg; Keynote opened normally (slides = 1, layouts = 23).

---

8. Paragraph Constraints and Structured Writes

Earlier replacements changed only characters and kept the paragraph count fixed. Appending a note increases the paragraph count; the experiment below shows that this is safe.

8.1 How Text Is Really Organized

A common mistake is to treat StorageArchive.text as one element per paragraph. In practice:

```
repeated text element count = 1
  elem[0] = 'Body level 1\nBody level 2\nBody level 3\nBody level 4\nBody level 5'
```

Paragraphs are separated by \n inside a string, not by repeated elements.

8.2 Paragraph Attribute Tables Are Empty

StorageArchive theoretically carries paragraph-indexed tables: table_para_data (6), table_para_style (5), table_para_starts (14). If these were indexed by character offset, adding or removing paragraphs would corrupt the offsets.

Full-sample measurement:

File	StorageArchive count	Max paragraphs	Objects with non-empty table_para_*
sample-deck.key	205	5	**0**
sample-deck.key	282	8	**0**

All 487 objects are empty; paragraph attributes are not stored at this layer. Adding or removing paragraphs in StorageArchive therefore does not corrupt internal indices.

8.3 Appending a Note

Appending one paragraph to page 1 of a real deck:

- notes 185 → 218 chars; object 497 → 561 bytes; entry payload lengths updated.
- **Keynote opened normally (slides = 21); presenter notes returned 218 chars and contains append = true**, with the new text at the end.

Conclusion: structured writes that add paragraphs are safe.

This supports a practical rule: new slides should carry presenter notes (content basis / design basis / talking points), and existing notes may only be appended to, never rewritten. The note subcommand implements append-only behavior.

8.4 Whole-Block Writes

Because paragraph counts can vary safely, a title or body can be replaced as a whole rather than by substring replacement. Follow the reference chain to the page's TSWP.StorageArchive and replace its text.

- set-text changed page 1 title from 8 to 15 characters; Keynote returned the new title.
- set-note replaced page 1 notes from 185 to 36 characters; Keynote returned the new content.

> set-note handles whole-block or substring replacement; note appends. For normal additions, use note.

---

9. Page-Level Switches: Skip and Collapse

The "skip slide" switch (hidden during playback but retained in the document) lives at:

KN.SlideNodeArchive.isSkipped (field 4, bool)

9.1 Three-Way Verification

Direction	Evidence
Read (Keynote → file)	A node with isSkipped=True (ident 7036107) corresponds exactly to Keynote's skipped of slide 12.
Write (file → Keynote)	Setting page 3 to isSkipped=True made Keynote report "skipped pages: 3,12".
Unskip (write 0)	Writing isSkipped=0 to page 12 made Keynote report "skipped pages: (none)".

9.2 A Real Trap: Booleans Must Be Explicitly Written as 0

The first unskip implementation deleted field 4. Keynote then reported -1700 cannot convert missing value to specifier and failed to load the document.

Byte-level diagnosis explained why:

```
Original deck's SlideNodeArchive field 4 presence:
  explicit =1 : 1
  explicit =0 : 43      ← non-skipped pages also write 0
  omitted     : 0
```

Keynote never omits this field. Even non-skipped pages explicitly write field 4 = 0 (proto2 optional presence semantics). After writing 0, all 44 nodes are =0, matching the original format; Keynote opened normally and reported "(none)".

Lesson: when rewriting a binary structure, "semantically equivalent omission" is not necessarily "byte-compatible"; copy the native representation.

9.3 Adjacent Switches: Collapse State

Two more switches live on the same node:

Field	Meaning	Native representation
isSkipped (4)	skip during playback	**explicitly write 0** (43/43 non-skipped nodes carry =0)
isCollapsed (5)	collapsed in outline view	**omit** (0 explicit occurrences)
isCollapsedInOutlineView (17)	collapsed in outline view	**omit** (0 explicit occurrences)

These fields behave differently: isSkipped is always written, while the two collapse fields are truly optional and absent by default. Writing the latter as explicit =1 still lets Keynote open the document (slides = 21), so they are safe to write, but they represent outline UI state, not playback.

This difference is the root of the lesson in 9.2: before changing a bool field, determine whether it is "always written" or "omitted by default".

---

10. Object Content: Tables, Charts, Builds, and Themes

10.1 Tables (TST.*): Text Is Writable

The real location of table numbers:

```
TST.TableModelArchive
  ├── number_of_rows / number_of_columns
  └── base_data_store.tiles.tiles[].tile → TST.Tile
        └── rowInfos[row].cell_storage_buffer
              ├── cell_offsets: little-endian uint16 array; item i = start of cell i
              ├── numeric cell: 36 bytes, value is a little-endian int32 at +12
              └── text cell: 24 bytes (points into stringTable)
```

The sample is 5 rows × 4 columns; row 2 is [2, 4, 4, 6]. Changing B2's 04 00 00 00 (offset +12) to 99 03 00 00 makes Keynote return **999.0**:

```
$ kn set-cell sample.key out.key 7031011 2 2 777
table 7031011 row 2 column 2: 4 → 777
→ Keynote: B2 = 777.0; C2 = 4.0 (unchanged); A2 = 2.0 (unchanged)
```

Adding/deleting rows is governed by more than TableModel.number_of_rows: TST.ColumnRowUIDMapArchive keeps three consistent indices — sorted_row_uids (sorted by UUID), row_index_for_uid, and row_uid_for_index. Appending without preserving the sort order makes Keynote refuse the file.

10.2 Charts (TSCH.*): Numbers Are Writable

A chart is a **TSCH.ChartDrawableArchive** (only super → TSD.DrawableArchive: geometry, wrapping, title/caption references). Its non-style data, TSCH.ChartNonStyleArchive, lives in **Index/ObjectContainer.iwa**; the presence of that entry indicates that the deck has charts (80 of 265 sample decks).

Neither TSCH.ChartDrawableArchive nor ChartNonStyleArchive stores values. Chart data is structurally the same as table data: values live in **TST.TableDataList + scaled integers in cell_storage_buffer**, so chart numbers can be edited with the same method. The TSCE formula engine only participates when cells contain formulas (22 TSCE.FormulaOwnerDependenciesArchive objects in the sample).

Actual location of chart data: it is inline in TSCH.ChartDrawableArchive.unity (type TSCH.ChartArchive) inside grid, not a separate TSCH.ChartArchive object:

```
chart_type: columnChartType2D
grid {
  row_name:    ["Region 1", "Region 2"]
  column_name: ["April", "May", "June", "July"]
  grid_row[{ value[{ numeric_value: 17 }, …] }, …]
}
```

Sample (2 rows × 4 columns):

```
$ kn charts sample.key
--- chart 7031018 (2 rows × 4 columns)
   rows: ['Region 1', 'Region 2']   columns: ['April', 'May', 'June', 'July']
   row 0: [17.0, 26.0, 53.0, 96.0]
   row 1: [55.0, 43.0, 70.0, 58.0]

$ kn set-chart sample.key out.key 7031018 0 0 100
chart 7031018: Region 1 / April 17.0 → 100.0
```

Keynote opened the result; exported PNG non-white samples went from 12,749 to 15,059, with 9,540 differing points (bar heights changed as 100 > 17). The written value also read back consistently.

10.3 Builds (KN.Build*): Readable and Writable

A page's animations are referenced by KN.SlideArchive.builds (2) and buildChunks (43):

- **KN.BuildArchive**: drawable (1) target, delivery (2), attributes (4) → BuildAttributesArchive (animationAttributes, eventTrigger, delayAutomatic*, etc.).
- **KN.BuildChunkArchive**: build (1), delay (3) **double**, duration (4) **double**, automatic (5) bool.

Sample (8 pages with animation):

```
page 1  build delivery=All at Once  effect=apple:bc-appear
        chunk delay=0.0 duration=0.5 automatic=False
```

```
page 9  build effect=apple:movie-start / apple:bc-appear
        chunk duration=1.0 automatic=False; duration=0.5 automatic=True
```

Writing duration 0.5 → 3.0 and automatic False → True read back consistently; Keynote opened normally (slides = 31).

> Trap: delay/duration are **double (wire type 1)**, not float (wire type 5). Writing them as float fails silently: the value does not change and no error is raised.

10.4 Master and Theme Colors

```
KN.ThemeArchive.super
  ├ color_presets[30]                          ← theme palette
  └ extension.gradient_fill_presets[] / image_fill_presets[]
KN.SlideStyleArchive
  ├ super                                      → TSS.StyleArchive
  └ slide_properties(11) → fill(1) → color(1)      ← page background
TSWP.ShapeStyleArchive
  └ super.shape_properties → fill → color          ← shape fill
TSWP.ParagraphStyleArchive
  └ char_properties → font_color / tsd_fill.color  ← text color (same value in two places)
```

TSP.Color is an inline submessage, not a top-level object. An earlier report counted zero because it inspected only the top-level object list. Recursing through the message tree finds **2,939** instances in the sample template and more than 2,000 in the master:

Location	Scale	Meaning
TSCH.* chart styles	~1,700	default chart series colors (style presets)
TST.* table styles	~650	table border/cell fill presets
TSWP.ParagraphStyleArchive	~140	paragraph text color
KN.ThemeArchive.color_presets	30	theme palette (white/gray/black/blue/cyan/green/yellow/orange/red/pink)
KN.SlideStyleArchive	1 per page	page background
TSWP.ShapeStyleArchive	1 per shape	shape fill/border/shadow

Color values are sRGB 0..1: model=rgb(1), rgbspace=srgb(1), and r/g/b/a are floats (wire type 5). AppleScript's color uses 0..65535 in **display space**: setting pure red {65535,0,0} on a P3 display makes Keynote store (1.0, 0.149, 0.0), the P3→sRGB conversion. **Write sRGB values directly.**

Three color paths verified with Keynote plus PNG pixel comparison:

Change	Target	Evidence
Text color	sample page font_color green → purple	PNG green disappeared; (6,51,251) appeared
Page background	page 2 SlideStyleArchive.fill.color white → red	page 2 became (252,59,47); page 1 stayed white
Shape fill	master style 734 red → green	all red blocks on 9 referencing pages turned green; pixel counts matched

A critical semantic trap: TSS.StylesheetArchive (569 styles in the master) is a style library, and many styles are not referenced by any page. Changing an unreferenced copy changes nothing; changing a referenced style changes every page that references it. That is the correct meaning of “edit the master/theme color”. The tool lists color instances with colors and reports how many pages share a style when using set-shape / set-bg.

Color commands:

```
# List all color instances (chart/table style presets are hidden by default; --all shows them)
kn colors deck.key

# Set one color by index (index from colors output)
kn set-color src.key dst.key 1943 '#FF3B30'

# Change page 2 background / page 3 shape 0 fill
kn set-bg    src.key dst.key 2 '#FF3B30'
kn set-shape src.key dst.key 3 0 '#FF3B30'
```

> Changing a referenced style affects all pages that use it (the tool prints the affected page count); changing an unreferenced copy has no visible effect. Both cases were tested.

10.5 Combined Audit Pipeline

audit combines the dimensions above in one read-only command:

```
$ kn audit sample-deck.key
total pages 21
  note compliance 21 pages | missing notes 0 | incomplete fields 0 | skipped 0 | builds 0 | tables 1

$ kn audit sample-animation.key
total pages 31
  note compliance 0 pages | missing notes 23 pages [2,3,4,...] | incomplete fields 8 pages
  skipped 0 | builds 8 pages page1(1 build/1 chunk); page2(1 build/3 chunks); ...
  tables 0
```

The same implementation can confirm that a compliant deck passes and expose 23 pages with missing notes, 8 pages with incomplete fields, and 8 pages with builds in a legacy deck.

---

11. Structural Editing: Slides and Shapes

11.1 Page Order Comes Only from ShowArchive.slideTree.slides

```
ShowArchive.slideTree (inline SlideTreeArchive)
  └ slides[] = repeated TSP.Reference → SlideNodeArchive
      └ slide(2) → SlideArchive
```

rootSlideNode is not set in practice; page order is determined entirely by the slides list. Therefore:

- **Delete slide** = remove the reference from slides, then cascade-delete objects.
- **Insert slide** = insert a new reference into slides.

11.2 Adding a Slide by Layout (Deep Copy)

Layouts are simply a set of SlideArchive objects with template=False and names such as “Title & Bullets”. A user page points to one via SlideArchive.template_slide.

add-slide <layout> [position]:

1. Find the layout by SlideArchive.name.
2. Deep-copy its entire closure: PlaceholderArchive, ShapeInfoArchive, StorageArchive, GroupArchive, and StandinCaptionArchive all get new identifiers; styles and themes remain shared.
3. Create a new SlideNodeArchive and insert it into slideTree.slides.

Verified: add-slide "Title & Bullets" produced 2 pages and deep-copied 24 child objects. Changing the new page's title left the old page unchanged.

> **Hard limit (settled)**: a page added or copied purely at the file layer is **not valid** in Keynote. The page count becomes 2, but setting current slide reports that the slide is not in the document, and PNG export is blank. The following were systematically ruled out: template_slide, placeholder structure, parent, style references, thumbnails (including real thumbnails), SlideNode UUID, template-specific fields, shared background objects, and .iwa ownership. Conclusion: Keynote requires an additional document-level registration for “new pages” that cannot be reliably forged at the file layer.
>
> **Structural knowledge gained**:
>
> 1. Each page's objects live in a separate iwa. Keynote creates Index/Slide-<page-ident>.iwa with 15 objects, one segment per object: SlideArchive ×1, PlaceholderArchive ×3, StandinCaptionArchive ×4, StorageArchive ×4, NumberAttachmentArchive ×1, GuideStorageArchive ×1, NoteArchive ×1. Slide.iwa contains only the original pages.

2. MessageInfo has five fields: type(1), **version(2)**, length(3), and **references(4/5)**. A segment with only type+length parses but Keynote rejects it. Adding version (01 00 05) and references still fails, so reference semantics (global ident versus local index) are not fully resolved.
3. ViewState is renamed on save: Index/ViewState.iwa becomes Index/ViewState-<ident>.iwa.
4. Decisive counterexample: copying Keynote's own page iwa byte-for-byte into another document (without even changing identifiers) makes the page visible, but current slide fails and export is blank. The missing piece is document-level registration, not page objects.

Deleting pages is unaffected: after deletion, the remaining 21 pages render normally (non-white samples 1944/1669/2594). Production recommendation: **delete with this tool; add/change layout through Keynote**. add-slide / dup-slide / change-layout are research commands and should not be used for delivery.

Recommended flow: let Keynote create the page, then use the tool for everything else:

```
kn add-slide deck.key out.key "Title & Bullets" \
      --title "New section" --body "Body" --after 3
```

Verified: --title works, page 2 renders correctly (non-white sample 926), and --after 1 inserts after page 1. If a layout name does not exist, all available layouts are listed.

Changing layout (--replace N): add a page with the target layout, migrate title/body, then delete the old page. Page order and content are preserved:

```
one page: 1 page, layout=Title & Bullets, title "Hand-written" preserved, render 468
three pages: layouts Quote / Title & Bullets / Blank No Page Number; render 1459 / 458 / 122
```

The recommended layout change is delete-old + add-new. Directly changing template_slide on an existing page makes Keynote refuse the file because the placeholder structure does not match the layout.

11.3 Group / Ungroup

TSD.GroupArchive has only two fields: super (ContainerArchive: geometry, parent, children…) and children (repeated Reference).

Three requirements for grouping:

1. Create a GroupArchive whose geometry is the union of the children.
2. Change each child's parent to the group; otherwise Keynote still looks at page level and cannot find it.
3. Keep the group in the z-order at the original position of the first child; appending it at the end raises background shapes above everything.

Verified with two of seven shapes on master page 1:

```
group   → page drawables 7 → 6; render non-white 1944; diff from original 0
ungroup → page drawables back to 7; render 1944; diff 0
```

Ungroup must also restore each child's parent to the page and insert the children at the group's z-order position. Moving children back without fixing z-order drops rendered non-white samples from 1944 to 629.

11.4 Deleting a Shape Must Cascade to Owned Children

Keynote deletes a shape together with its owned objects:

```
TSWP.ShapeInfoArchive
  ├ owned_storage / deprecated_storage → TSWP.StorageArchive (text container)
  └ super.super.title / caption        → TSD.StandinCaptionArchive
```

Removing only the reference from owned_drawables / drawables_z_order while leaving the object segments makes Keynote refuse the file. Deleting owned_storage, title, and caption as well works. style (shared) and parent (pointing to the page) must not be deleted.

11.5 Page-Precise Text Color: Character-Level Override

Text color normally lives on the shared ParagraphStyleArchive (one paragraph style is shared by 134 text objects in the master). To change only one page without affecting paragraph layout, use a **character-level override**:

1. Create a new TSWP.CharacterStyleArchive with a new identifier; copy the page's paragraph style properties (font, size) into char_properties, overriding only font_color and tsd_fill.color.
2. super.name must be set (for example “Custom color #FF00FF”), and the style must be registered in the shared TSS.StylesheetArchive.styles.
3. Set the storage's table_char_style.entries to [{character_index: 0, style}]; do not write a closing entry.
4. Leave the paragraph style reference unchanged; the shared style is not modified.

Three tested traps:

Trap	Symptom	Fix
super.parent points to a paragraph style	style silently ignored; no color	parent must be another CharacterStyleArchive; a different type does not work
style not registered in stylesheet.styles	not rendered	append the new style to the shared styles[]
closing entry with character_index == len(text)	entire char_style dropped	write only {0, style}; the range extends to the end

Trap	Symptom	Fix
copying only local char_properties	text box not rendered; paragraph styles often carry only font_size, with font/color inherited	merge the inheritance chain through super.parent, then override color

Text boxes are supported: which can be title/body or a drawable path (2, 3/1 inside a group). All 53 colored character styles in the real file are attached to ShapeInfo (text boxes), so character-level color is intended for text boxes:

```
$ kn set-text-color sample-deck.key out.key 1 2 '#FF00FF'
page 1 shape 2 text color → #FF00FF
  new character style ident=7027964 (character override; shared style unchanged)
→ exported PNG: 3211 pixels changed, 1730 magenta
```

Partial color: --substr text changes the first occurrence; --range a:b changes [a,b) (1-based). Range entries are written as "insert a new style at start, restore the original at end; omit end when it equals text length". Entries must start with character_index = 0; writing only {1, style} has no effect until {0, default} is added.

Form	PNG result
--substr character (1 of 3)	1104 pixels changed, 587 magenta
--range 1:2 (first character)	751 pixels changed, 406 magenta
whole run (default)	3211 pixels changed, 1730 magenta

Two-page sample:

```
page 1 (blank): magenta 0
page 2 (changed): magenta 1655
page 3 (control): magenta 0
shared paragraph style 1611008 font_color remains (0,0,0)
new character style: name="Custom color #FF00FF", parent=1611307, color=(1,0,1)
```

When Keynote changes color for an entire paragraph, it copies a paragraph style variant (para entries: [(0, 7028480)], char entries: None), whereas character styles are mainly for local emphasis (design-spec.key has 53 real references). Both paths are supported: --char (default) and --para.

11.6 Verified Results

Operation	Result
del-slide 5 (master)	22 → 21 pages; Keynote opens
dup-slide 1	22 pages; opens; shares drawables with the original

Operation	Result
add-slide "Title & Bullets"	1 → 2 pages; 24 objects deep-copied; new title does not affect the old page
del-shape 1 3	page 1 shapes 3 → 2; Keynote opens
dup-shape 1 0	page 1 shapes 3 → 4; Keynote opens
set-text-color 2 title #FF00FF	page 2 has 1655 magenta pixels; pages 1/3 have 0; shared paragraph style remains black
set-text-color 1 2 #FF00FF (text box)	3211 pixels changed, 1730 magenta; other elements unchanged
set-text-color 1 2 #FF00FF --substr character	target character only: 1104 changed, 587 magenta
set-text-color 1 2 #FF00FF --range 1:2	first character only: 751 changed, 406 magenta
set-note ... --replace second paragraph	First. Second. → First. Replaced.; the rest is preserved
set-note (empty / existing note)	0 → 7 chars; 7 → 8 chars; Keynote reads the second write

11.7 Diagnostic Boundaries for Damaged Files

Three damage modes:

Damage	verify behavior
Truncated Zip / empty file	readable error: "not a valid .key file — Zip structure damaged or truncated"
One .iwa byte changed	graceful degradation: × Index/Document.iwa: unsupported wire type 7; the other 30 iwa files parse
Other commands on a bad Zip	capture BadZipFile / FileNotFoundError / PermissionError without a traceback

Design principle: file-level errors produce readable conclusions; segment-level errors affect only that file and allow the rest to be counted. This distinguishes "whole file broken" from "one iwa broken, the rest usable".

11.8 Table Row Insert/Delete: Three UID-Map Traps

add-row / del-row tests:

Observation	Conclusion
Change only TableModel.number_of_rows	Keynote ignores it; row count unchanged
Change only Tile.rowInfos	opening reports a missing required field
Append a UID to ColumnRowUIDMapArchive without sorting	**file refused**

Correct method (three places in sync):

1. Insert the new UUID into sorted_row_uids by **upper ascending**, then lower.
2. row_index_for_uid: increment original values at or after the insertion position by 1; put the insertion index at the sorted position.
3. row_uid_for_index: insert the sorted index of the new UID at the insertion position.
4. Insert into Tile.rowInfos and reorder all tile_row_index values.

Verified: Keynote reports rows=6; the new row copies the source row (A3=2.0; A4 is the old row 3). After deletion Keynote reports rows=4, and later rows shift correctly.

Add column: append a 24-byte empty text cell to every row's cell_storage_buffer (05 03 + 8 zero bytes + 08 10 02 00 + 4-byte text ID + 05 00 00 00 01 00 00 00), increment cell_count and number_of_columns, and update the three column indices. Verified: Keynote reports 5 columns; the new column is empty.

Delete column: Keynote itself changed 14 objects when deleting a column; after aligning each one, del-col works:

1. Recompute cell_offsets as uint16 and fill extra slots with 0xFFFF (retaining old values makes Keynote refuse the file).
2. In the legacy cell_storage_buffer_pre_bnc, delete one 12-byte cell per row; fill cell_offsets_pre_bnc with 0xFFFF as well.
3. In HeaderStorageBucket, delete one header, renumber indices consecutively, and redistribute widths by the total.
4. Update the row header's numberOfCells to the new column count.
5. In ColumnRowUIDMapArchive.column_uid_for_index, delete the removed column and decrement every value greater than its position.
6. In the TableDataList string table, remove keys referenced by the deleted column (text-cell offset 12 int32); do not renumber the remaining keys.
7. In the formula engine, set FormulaOwnerDependenciesArchive.spanning_column_dependencies.total_range_for_table.bottom_right_co to the new column count − 1.

Verified by deleting column 3: Keynote returns rows=5 cols=3, A2=2, B2=4, C2=6 — the data after the deleted column.

> Lesson: adding a column only provides reverse-engineering hints. 0xFFFF filling, the −1 remapping in column_uid_for_index, and string-table cleanup were all found by comparing against Keynote's own column deletion.

11.9 Image Replacement

TSD.ImageArchive has data, thumbnailData, and originalData as TSP.Reference values whose identifier is the final path segment of the Data/ filename:

```
data: 33012  →  Data/0F6230E7-...-33012.jpg  (9.1 MB)
data: 33005  →  Data/8A076B9D-...-33005.pdf
```

set-image replaces the file directly and updates naturalSize / originalSize. Replacing a 9.1 MB JPG with an 800×600 magenta test image opened normally in Keynote and exported 44,304 magenta pixels.

11.10 Hyperlinks

TSWP.HyperlinkFieldArchive has two key fields:

```
super.text_attribute_uuid_string: "FFF9DB61-..."
url_ref: http://www.google.com          ← external link
url_ref: ?slideid=43E29214-...          ← in-document jump
```

links lists all links; set-link changes url_ref.

11.11 Slide Transitions

A transition is not a standalone object; it is inline in SlideArchive.transition (KN.TransitionArchive):

```
transition {
  attributes {
    animationAttributes {
      animation_type: "Transition"
      effect: "apple:magic-move-implied-motion-path"   ← none / apple:dissolve / ...
      duration: 2.0
    }
  }
}
```

transitions lists each page; set-transition changes the effect and duration. Changing page 2 from magic-move to apple:dissolve + 1.5 s read back consistently, passed verify, and opened in Keynote.

11.12 Adding Builds

KN.BuildArchive has five fields:

```
drawable: { identifier: 7030297 }       ← target object
delivery: "All at Once"
duration: 0.0
attributes {
  eventTrigger: 1
  animationAttributes {
    animation_type: "In"
    effect: "apple:bc-appear"           ← effect name
    duration: 1  delay: 0  random_number_seed: ...
```

```
  }
}
chunk_id_seed: 1
```

KN.BuildChunkArchive points to the build and carries its own build_chunk_identifier (UUID pair).

To add one: copy an existing BuildArchive, replace drawable and effect; copy a BuildChunkArchive, point it at the new build, regenerate UUIDs; append both references to SlideArchive.builds / SlideArchive.buildChunks.

Verified by adding apple:bc-appear to page 2 object 0: builds-list shows 3 builds, verify passes, Keynote opens (slides = 2).

> Limitation: file-level validity and "Keynote opens" are confirmed, but whether the animation actually plays was not checked in the Keynote UI. Run a manual slideshow before delivery.

11.13 Style Library Editing

Style objects carry super.name and super.style_identifier; changing one style affects every text object that references it.

styles lists all styles (ident / type / name / style_identifier / key properties):

```
  1611294  character  Red (bold)       bold=True color=#C82505
  1611176  character  Link             character-style-hyperlink
  1611213  paragraph  –                chart-0-paragraphStyle_0   size=13 color=#000000
```

set-style changes one property (font_size / font_color / bold / italic):

```
$ kn set-style sample-deck.key out.key 6882870 font_color=#FF00FF
style 6882870 "Quote": font_color None → #FF00FF
→ exported PNG: 3653 magenta (same as whole-run set-text-color)
```

> Division of labor: set-style is batch (all pages referencing the style change together); set-text-color is page-precise (new CharacterStyle, shared styles untouched).

11.14 Performance

File	Size	Parse	Objects	Per object	Write (incl. process start)
sample template, 1 page	0.4 MB	22 ms	1360	16 µs	168 ms
master template, 22 pages	1.0 MB	31 ms	2635	12 µs	183 ms
design spec, 42 pages	1.5 MB	57 ms	5681	10 µs	252 ms
sample chart deck, 1 page	0.5 MB	23 ms	1361	17 µs	—

Parsing scales linearly with object count; extrapolating from 42 pages / 57 ms, 1,000 pages is about 1.4 s to parse and 3–5 s to write, not a bottleneck.

11.15 Diagnostic Lesson: “Will Not Open” Is Not Always Corruption

Several “Keynote will not open” cases during debugging were actually silent AppleScript open POSIX file failures (–1719 / –1712). open –a "Keynote Creator Studio" <file> opened them normally. Judge real corruption with verify + open –a + field read–back together.

---

12. Real–Device Verification: 23 Edits

Scenario	Change	Result
Equal–length replacement	layout name _BlackBackground → _WhiteBackground (34→34 bytes)	opens; new name read back
Variable–length replacement	_BlackBackground → _SolidBlackBackground (34→37 bytes)	opens; new name read back
Cross–file batch	footer in 17 .iwa files: © Sample copyright → © Sample copyright 2026 (+5 bytes each)	17 files, 17 occurrences changed; opens and reads new footer
Append paragraph	page 1 notes 185→218 chars (+1 paragraph)	opens; presenter notes returns 218 chars including the append
Skip page	page 3 isSkipped=1	Keynote reports skipped pages 3,12 (new 3 plus native 12)
Unskip	page 12 set to isSkipped=0 explicitly	Keynote reports skipped pages “(none)”
Batch skip	2–4,6 in one write	Keynote reports 2,3,4,6,12
Rewrite body	page 1 title 8→15 chars	Keynote returns the new title
Overwrite note	page 1 notes 185→36 chars	Keynote returns the new content
Replace media	same–named Data/mt–*.jpg bytes	opens (slides=1, layouts=23)
Table text	"metric" → "comparison metric" (+6 bytes in Index/Tables/DataList.iwa)	page 17 returns the new text / recall@8 / 0.929
Build timing	duration 0.5→3.0, automatic False→True	reads back; opens (slides=31)
Combined audit	compliant and legacy decks	compliant all green; legacy reports 23 pages missing notes and 8 pages with builds
Table number	row 2 column 2 0.929→0.777	Keynote returns 0.777
Text color	sample page green→purple	green disappears, purple appears in PNG; AppleScript read–back is cached and unreliable
Page background	page 2 white→red	page 2 becomes (252,59,47); page 1 stays white

Scenario	Change	Result
Shape fill	shared style 734 red→green	9 referencing pages turn green; other colors unchanged
Delete slide	master page 5	22 → 21 pages; opens
Duplicate slide	page 1 duplicated after itself	22 pages; new page shares drawables
Add slide by layout	add-slide "Title & Bullets"	1 → 2 pages; 24 objects deep-copied; old page unchanged
Delete shape	page 1 shape 3 (cascade storage/caption)	shapes 3 → 2; opens
Duplicate shape	page 1 shape 1	shapes 3 → 4; opens
Page-precise text color	new CharacterStyle on table_char_style	target page 1655 magenta; control 0; shared style unchanged
Text-box color	drawable path 2 (including 3/1)	3211 pixels changed, 1730 magenta
Insert shape	dup-shape --offset 0,250	opens; render 1459→1648; independent copy
Group / ungroup	group / ungroup	render 1944; diff from original 0
audit --fix	audit --fix --out	1 page missing note → 0
Partial color	--substr / --range	character-level change; pixel count matches character count
Note substring replace	set-note --replace	only target substring replaced; rest preserved
Rewrite existing note	empty note written, then whole-block replacement	Keynote returns second content

A Notable Trap

The first variable-length implementation updated only MessageInfo.length and the ArchiveInfo prefix. Keynote could open the file (layouts=23), but the layout name read back as missing value. The cause: the length of the protobuf field wrapping the string **inside the object** was not updated. Recursive descent and layer-by-layer recalculation fixed it. Lesson: “opens” does not equal “correct”; read back the changed field.

---

13. Operability Boundaries

Operation	Status	Notes
Unpack/read structure	Verified	Zip + IWA + Snappy

Operation	Status	Notes
Semantic object identification	Verified	1,360/1,360 parsed; 631 type mappings
Extract real text	Verified	by object; footers/body/layout names
Page outline	Verified	title/body/notes on a 21-page deck
Equal-length replacement	Verified	no length fields touched
Variable-length replacement	Verified	length chain recalculated
Cross-file batch replacement	Verified	17 files / 17 occurrences
Append presenter notes	Verified	page-level append; Keynote reads back
Skip/unskip page	Verified	isSkipped (4); three-way test + range expressions
Rewrite title/body	Verified	reference chain → StorageArchive.text
Rewrite existing note	Verified	set-note whole block; --replace substring
Note compliance audit	Verified	21-page deck fully compliant
Replace media	Verified	same-name bytes; no metadata sync
Table text	Verified	Index/Tables/DataList.iwa string table
Table numbers	Verified	scaled integer at cell_storage_buffer +12; reads 0.777
Chart numbers	Same mechanism as tables	structurally identical data source; not separately tested
Build read/write	Verified	builds; set-build duration/automatic
Enumerate colors	Verified	recursive inline TSP.Color (2,939 in sample)
Set color by index	Verified	set-color; Keynote + PNG pixels
Page background	Verified	set-bg; page 2 white→red; page 1 unaffected
Shape fill	Verified	set-shape; shared style reports affected pages
Page-precise text color	Verified	new CharacterStyle on table_char_style; whole run / --substr / --range / --para-index for title, body, text boxes
Combined audit	Verified	audit combines notes/skips/builds/tables
Delete/duplicate slide	Verified	del-slide / dup-slide; order from slideTree.slides
Add slide by layout	Verified	add-slide; deep-copies layout closure; content independent

Operation	Status	Notes
Delete/duplicate shape	Verified	del-shape / dup-shape; deletion cascades storage/caption
Page-precise text color	Verified	set-text-color supports title/body and text-box paths; character override by default
Rewrite existing note	Verified	set-note; whole block or --replace

Recommended path:

- **Text changes** (title, body, footer, layout name, notes) → bin/kn; fast, scriptable, batchable.
- **Structural changes** (add/delete page, change layout, insert shape) → AppleScript driving Keynote.

---

14. Risks and Limitations

1. **The schema is version-aligned**: this work uses v14_5 definitions (Keynote 14.5). Type numbers may change across major versions; update the schema when that happens.
2. **Cross-version behavior may break**: always test a copy in Keynote after editing.
3. **Always keep the original.**
4. **Only existing string content can be changed**: replacement cannot create new objects from nothing, because the reference graph would become inconsistent.
5. **Write-back uses real Snappy compression**: iwa_wrap performs greedy LZ77 compression and a round-trip self-check, falling back to literals on failure. The modified DocumentStylesheet was 47,004 bytes versus 47,858 native; an older literal-only path inflated it to 305,744 bytes.
6. **Changing a style is not changing one page**: colors hang off shared styles; changing one affects every referencing page. The tool prints the affected pages.

---

15. Appendix: Tool and Usage

bin/kn (Rust implementation; the read/write core does not require an external schema, and semantic commands use protobuf plus the built-in schema).

```
# Container and validation
kn list   <file.key>
kn verify <file.key>

# Byte-level text search without a schema
kn text   <file.key> [keyword]

# Semantic layer
kn types   <file.key>           # object counts by type
kn texts   <file.key> [keyword]   # real text and owning objects
```

```
kn outline <file.key>            # page outline: title/body/notes

# Note compliance (requires content/design basis by default)
kn check-notes <file.key> [required fields...]

# Append one paragraph to notes (existing content preserved)
kn note      <src.key> <dst.key> <page> <text to append>

# Rewrite notes (whole block; --replace changes a substring)
kn set-note <src.key> <dst.key> <page> <new note text>

# Rewrite a whole title/body
kn set-text <src.key> <dst.key> <page> <title|body> <new text>

# Skip / unskip (page expressions: 3 / 1-3,7 / all)
kn skip      <src.key> <dst.key> <page expression> [--off]

# Object content
kn audit     <file.key>              # combined audit
kn builds    <file.key>              # list builds by page
kn set-build <src> <dst> <chunk ident> [duration=2.5] [automatic=true]

# Colors (TSP.Color is an inline submessage)
kn colors    <file.key>              # list color instances
kn set-color <src> <dst> <index> <#RRGGBB|r,g,b> [--alpha a]
kn set-bg    <src> <dst> <page> <#RRGGBB>
kn set-shape <src> <dst> <page> <shape path> <#RRGGBB>
kn set-text-color <src> <dst> <page> <title|body|shape path> <#RRGGBB> \
                                [--substr text|--range a:b] [--para]

# Structural editing
kn add-slide <src> <dst> <layout name> [position]
kn del-slide <src> <dst> <page expression>
kn dup-slide <src> <dst> <page>
kn del-shape <src> <dst> <page> <shape path>
kn dup-shape <src> <dst> <page> <shape path>

# Replacement (equal length by default; --resize allows growth; --all across .iwa)
kn edit    <src.key> <dst.key> <iwa name|-> <old string> <new string> \
                              [--resize] [--all]
```

Sample output:

```
$ kn types sample-deck.key
total objects 1360 (recognized 1360 / unknown 0)

$ kn outline sample-deck.key
21 pages

--- Page 1 ---
  title: Sample deck
  body: A real build-vs-buy trade-off
  notes: present (185 chars)
...
summary: 21 pages; 21 have presenter notes

$ kn texts sample-deck.key copyright
17 text objects contain the keyword 'copyright'
  [  6879323] TemplateSlide.iwa  1 paragraph  '© Sample copyright'

$ kn edit sample-deck.key out.key - \
      "© Sample copyright" "© Sample copyright 2026" --resize --all
wrote out.key (509136 bytes; 17 .iwa files modified, 17 occurrences replaced)
```

Operating discipline: locate with texts → edit a **copy** (with read-back verification) → open in Keynote → replace the original only after confirmation.

Schema Definition

The official protobuf schema v14_5 provides the object-type mapping. The tool itself does not require an external runtime schema package.

---

16. Methodology

Three independent evidence lines agree:

1. **Artifact analysis**: structural analysis of document artifacts (61 nodes / 60 edges, integrity_contradictions: 0).
2. **Custom decoder**: Snappy, varint, protobuf node conversion/re-encoding, and segment splitting are all implemented independently. The segment model was first falsified by an incorrect hypothesis, then confirmed by a full parse (31 files / 1,354 segments / 1,360 objects).
3. **Official schema**: the official serialized field definitions independently confirm the segment structure; the v14_5 definitions supply object-type mappings and parse 1,360/1,360 objects.

The acceptance criterion is always opening the result in Keynote 14.5: equal-length, variable-length, and cross-file batch edits all pass.